

Space

Product Requirements and Engineering Specification

Prepared for the Space product and engineering team

11 July 2026

Version 1.0

Implementation baseline

Internal

Contents

Top-level sections in this implementation baseline

1	Document purpose
2	Executive summary
3	Product definition
4	Glossary
5	Scope hierarchy and isolation model
6	Product information architecture
7	Core user journeys
8	Onboarding and computer bootstrap specification
9	Workspace management requirements
10	Project management requirements
11	Git engine specification
12	Git history and graph specification
13	Changes and intent layer
14	GitHub integration specification
15	Terminal specification
16	Environment and service adapters
17	Activity specification
18	Automation specification
19	Agent and computer interaction specification
20	Electron architecture
21	Workspace Process Runner
22	IPC contract
23	Local data architecture
24	Credential and secret storage
25	Security and threat model
26	Error handling and recovery
27	Performance requirements
28	Accessibility and interaction requirements
29	Privacy, analytics, and logging
30	Cross-platform implementation details
31	Packaging, updates, and release engineering
32	Recommended repository structure
33	Coding standards
34	Testing strategy
35	Quality gates and definition of done
36	Implementation programme
37	Epics and acceptance criteria
38	UI state checklist
39	Prohibited implementation shortcuts
40	Architecture Decision Records required
41	Open product questions requiring explicit decisions
42	Technical references
43	Final implementation directive

1 Document purpose

This document is the implementation baseline for **Space**, a cross-platform Electron desktop application that removes the setup, identity-switching, Git, GitHub, runtime, terminal, and project-management friction that prevents people from building software.

The document is deliberately prescriptive. The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are requirements keywords:

MUST / MUST NOT: mandatory for the relevant release.

SHOULD / SHOULD NOT: expected unless a written architecture decision record explains the exception.

MAY: optional implementation choice that must not break a mandatory requirement.

The team must not infer product behaviour from mock data or visual design alone. Every screen that represents Git, GitHub, a terminal, an installed tool, a process, a credential, or a project state must be backed by real local state and real commands.

1.1 Document control

Working product name: Space

Document: Product Requirements and Engineering Specification

Version: 1.0

Status: Approved implementation baseline, subject to ADRs listed in this document

Target platforms: macOS and Windows

Primary application framework: Electron with TypeScript

Source of truth: This document plus approved Architecture Decision Records

Intended readers: Product, design, Electron, frontend, systems, QA, security, and release engineering

1.2 Required reading order

New team members should read the following sections before writing code:

Product definition and non-negotiable principles.

Scope hierarchy and workspace isolation model.

Security boundaries.

Electron process architecture.

Workspace Process Runner.

Functional requirements for the feature being implemented.

Testing and definition of done.

2 Executive summary

Space turns a Mac or Windows computer into a ready-to-build development environment through a clean, minimal onboarding experience. It detects what is already installed, proposes only the missing essentials, uses trusted operating-system package managers to install them, verifies the result, and records every change.

After setup, the user creates any workspace by entering a name. A workspace is a named developer context, not a predefined category such as Work or Personal. It groups the projects, terminals, Git identity, GitHub account, cloud CLI profiles, environment variables, secrets, activity, and automation that belong together.

The user can then:

Create a new project.

Add an existing folder.

Clone a repository.

Initialise Git when a folder is not yet a repository.

Generate an appropriate .gitignore safely.

Create an initial commit.

Publish the same local project to GitHub, choosing the owner, repository name, visibility, and remote.

Work with real branches, history, changes, pull requests, checks, Actions, releases, and deployments.

Switch workspaces without repeatedly logging services in and out.

Use an integrated terminal in which normal CLIs behave naturally, while their configuration and credentials remain scoped to the active workspace where technically possible.

Space is not a replacement for Git, GitHub CLI, package managers, runtimes, or cloud CLIs. It orchestrates those proven tools through a coherent interface and an explicit workspace context.

The strongest product promise is:

Select a workspace and Space gives every project the correct identity, tools, credentials, terminals, Git state, GitHub context, and automation without the repeated friction of manual setup and account switching.

3 Product definition

3.1 Vision

Software creation should begin with the idea, not with hours of environment setup, account switching, terminal configuration, package-manager errors, or uncertainty about which identity will be used to push and deploy.

Space makes the computer feel ready for the context the user selected. It gives beginners a clear visual path while preserving the real terminal, real files, real Git repository, and real service CLIs underneath.

3.2 Problem statement

Modern development work is fragmented across:

Operating-system package managers.

Git configuration.

Multiple GitHub identities and organisations.

SSH keys and HTTPS credentials.

Node and package-manager versions.

Cloud providers and their CLI profiles.

Editors and integrated terminals.

Local projects and remote repositories.

Pull requests, checks, releases, and deployments.

Each tool works, but the user must remember which account, configuration, project, branch, runtime, and credential is active. This creates repeated friction and serious mistakes, including:

Committing with the wrong email address.

Pushing to the wrong GitHub account or organisation.

Deploying to the wrong cloud project.

Running a project with the wrong Node version.

Mixing client credentials with personal projects.

Losing local work because Git was never initialised.

Creating an empty GitHub repository separately and then struggling to connect it.

Installing duplicate or conflicting developer tools.

Avoiding the terminal entirely because the setup feels unsafe or confusing.

3.3 Product promise

Space MUST provide a path from a fresh computer to a running, version-controlled, optionally published project without requiring the user to manually edit shell profiles, create a remote repository in a browser, or understand every command involved.

Space MUST also keep the real commands observable. A user who wants technical detail can inspect the exact command, output, exit status, environment category, and resulting state for every operation.

3.4 Target users

3.4.1 Primary

Vibe coders and new developers who can describe and edit software but do not want to become environment experts first.

Founders and designers who work across multiple products, clients, or organisations.

Developers with several GitHub and cloud identities.

Students who need a safe, understandable development setup.

3.4.2 Secondary

Experienced developers who want reproducible context switching and a consolidated Git/GitHub workflow.

Teams that want consistent onboarding and project bootstrap.

Educators helping learners begin development without spending a full session on setup.

3.5 Product principles

Ask for the minimum. A new workspace initially requires only a name.

Use the real tool. Git data comes from Git CLI; GitHub operations come from gh; terminals are real PTYs.

Do not overwrite global user state. Scope processes through environment and explicit configuration paths rather than repeatedly rewriting global configuration files.

Detect before installing. Never reinstall or replace a healthy tool without a deliberate, visible action.

Make every change observable. Show a simple explanation by default and complete technical detail on demand.

Prefer reversible operations. Create receipts, checkpoints, backups, and recovery guidance.

Separate context, not the entire operating system. A workspace is a managed process and configuration boundary, not a virtual machine or hostile-code sandbox.

Progressive disclosure. Beginners see intent and outcome; experienced users can inspect commands, logs, diffs, refs, and configuration.

No simulated production state. Fixtures are for tests and demos only. Product screens use real data.

Local first. Core project, Git, terminal, and activity capabilities work without a Space backend.

Secure by construction. The renderer never receives unrestricted shell, filesystem, credential, or native-module access.

Cross-platform parity. macOS and Windows flows should feel equivalent even when the underlying package managers and shells differ.

3.6 Non-goals for the first production release

Space V1 MUST NOT attempt to:

Replace VS Code, Cursor, Xcode, Android Studio, or other full editors.

Replace Git with a custom repository implementation.

Replace GitHub with a proprietary remote source-control service.

Guarantee hostile-code isolation between workspaces.

Repackage every cloud CLI authentication system.

Install every possible developer tool during onboarding.

Run a permanent privileged daemon.

Rewrite the user's existing global shell, Git, GitHub, or cloud configuration without explicit consent.

Support Linux desktop as a release target in V1.

Provide background GitHub webhook automation while the desktop app is closed; that belongs to the later GitHub App phase.

4 Glossary

Space: The Electron desktop application.

Workspace: A user-named developer context containing projects, terminals, service configurations, Git identity, variables, activity, and automation.

Project: A local folder registered in a workspace. It may or may not initially be a Git repository.

Machine scope: Tools installed once for the computer and shared safely, such as Git, GitHub CLI, Volta, package-manager binaries, and editors.

Workspace scope: Identity, configuration, credentials, terminal state, service profiles, variables, permissions, and project membership associated with one workspace.

Project scope: Repository, files, dependency manifests, local dependencies, runtime pin, project variables, and remote association.

Workspace Process Runner: The privileged service that constructs a workspace-aware environment and executes approved commands.

Operation: A structured action requested by the UI or automation, such as initialise Git, install a tool, create a repository, or run a test.

Receipt: A durable record of an operation, including intent, steps, redacted command output, exit status, resulting changes, and recovery guidance.

Service adapter: Code that detects, configures, authenticates, and reports the state of an external CLI or service inside a workspace.

Checkpoint: User-facing wording for a real Git commit. The underlying Git terminology remains visible in technical details.

Intent group: A human-understandable grouping of related file changes derived from the real diff.

Trusted project: A project for which the user has allowed dependency installation, scripts, and process execution.

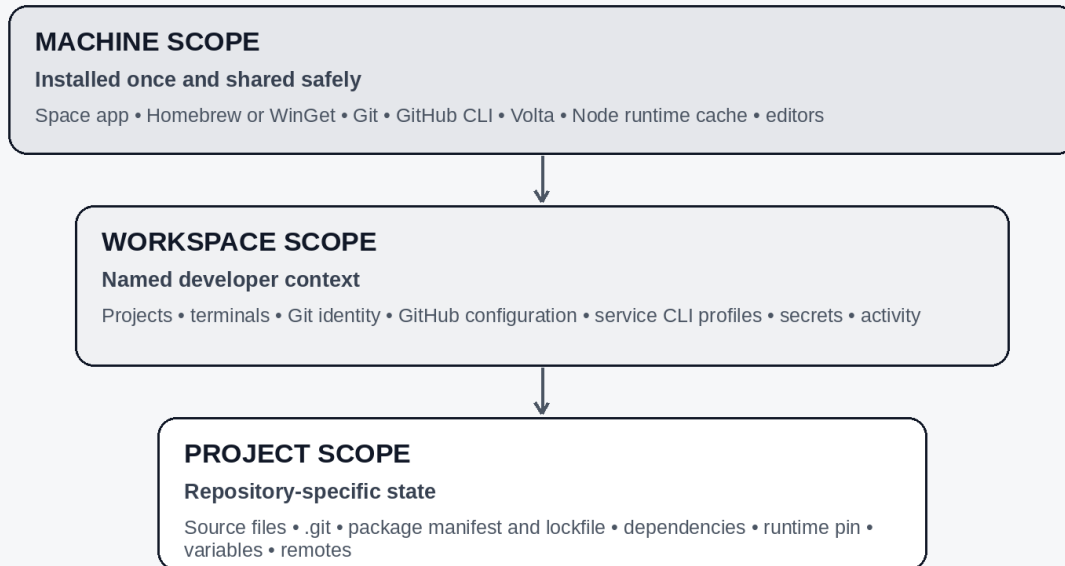
PTY: Pseudoterminal used to run an interactive shell or CLI authentication flow.

ADR: Architecture Decision Record documenting a significant decision or approved deviation.

5 Scope hierarchy and isolation model

Space Environment Scope

Keep machine tools, workspace identity, and project state in their correct layer.



5.1 Mandatory scope boundaries

5.1.1 Machine scope

The following are normally installed once and shared:

Space itself.

Homebrew on macOS, where required and supported.

WinGet/App Installer on Windows, where available.

Git.

GitHub CLI.

Volta.

Downloaded Node runtime versions and tool caches managed by Volta.

Editors.

Optional machine-level tools selected by the user.

The team **MUST NOT** create a separate Homebrew installation per workspace. The team **MUST NOT** duplicate Git or GitHub CLI binaries per workspace without a documented compatibility reason.

5.1.2 Workspace scope

Each workspace owns or references:

Name, icon, and optional default project location.

Registered projects and ordering.

Workspace Git configuration file.

Git author name, email, and signing settings.

GitHub CLI configuration directory or workspace-specific token reference.

- SSH identity configuration where enabled.
- Cloud and service CLI configuration directories/profiles.
- Workspace environment variables.
- Secure-secret references.
- Terminal sessions and shell history metadata.
- Trusted-project permissions.
- Activity events.
- Automations.
- Cached GitHub metadata.
- Default toolchain policy for newly created projects.

5.1.3 Project scope

Each project owns:

- Local folder path and filesystem identity.
- Git repository and worktree state.
- Dependency manifests and lockfiles.
- Project-local dependencies such as `node_modules`.
- Runtime pins, for example Volta metadata, `.nvmrc`, `.node-version`, or tool-specific files.
- Project-specific environment-variable references.
- Remote repository mapping.
- Project scripts and detected framework metadata.
- Trust state and permissions.

5.2 Accurate isolation promise

Space workspaces provide **context isolation** for processes launched through Space. They do not create a kernel-level security boundary.

The product copy and documentation **MUST NOT** say that every workspace is literally a separate computer. The acceptable user-facing explanation is:

Space keeps each workspace's projects, accounts, terminals, credentials, and settings together, so tools opened through Space use the context you selected.

Processes started outside Space are not automatically changed. Existing terminals, editors, browsers, and background processes retain the environment with which they were launched.

5.3 Environment precedence

The final environment for a process **MUST** be assembled in this order, where later layers may override approved keys from earlier layers:

- Sanitised operating-system base environment.
- Space machine-tool paths.
- Workspace tool paths and configuration variables.
- Workspace non-secret variables.
- Project runtime and project non-secret variables.
- Approved secure-secret values.
- Temporary operation-specific variables.

The runner **MUST** maintain an allow/deny policy for inherited variables. Sensitive host values that do not belong to the workspace **MUST NOT** be inherited blindly.

5.4 Configuration strategy

Space **MUST** prefer official configuration-directory or profile mechanisms. Examples include:

Git: workspace-specific configuration through `GIT_CONFIG_GLOBAL` or command-scoped configuration.

GitHub CLI: `GH_CONFIG_DIR`, with a tested secure-token fallback when operating-system credential-store behaviour requires it.

Google Cloud CLI: `CLOUDSDK_CONFIG`.

AWS CLI: named profiles and workspace-specific config/credentials paths where applicable.

Other services: official environment variables, profile flags, or configuration roots.

Space **MUST NOT** continuously rewrite `~/.gitconfig`, the user's normal GitHub CLI directory, or normal cloud configuration directories when switching workspaces.

5.5 Service-adapter isolation tiers

Every service adapter **MUST** declare its isolation tier:

Tier 1 — Official configuration-directory environment variable: Fully workspace-scoped through a documented config root.

Tier 2 — Official named profile/account mechanism: Space selects the workspace profile for each process.

Tier 3 — Process-specific HOME/XDG shim: Allowed only after compatibility and security tests. Must not affect processes outside Space.

Tier 4 — No reliable process-level isolation: Space discloses the limitation and requires explicit account selection, a container/VM mode, or does not support isolation for that service.

The product **MUST NOT** claim universal isolation for every CLI. Each adapter's capability is explicit and testable.

5.6 GitHub authentication caveat

`GH_CONFIG_DIR` isolates GitHub CLI configuration files, but authentication tokens may interact with the operating system credential store. Before GitHub multi-account support is considered complete, the team **MUST** run a P0 technical spike with at least two GitHub accounts on both macOS and Windows.

The implementation **MUST** support this fallback design:

Store a workspace's GitHub token in the operating-system credential store under a Space-owned service/account identifier.

Store only the token reference in the local database.

Inject `GH_TOKEN` into the relevant process at runtime.

Redact the token from logs, receipts, crash reports, and child-process diagnostics.

The selected strategy must be recorded in ADR-002 after the spike.

6 Product information architecture

6.1 Primary navigation

The application shell **MUST** contain:

Workspace switcher.

Workspace home.

Projects.

Changes.

History.

GitHub.

Activity.

Automations.

Terminals.

Environment.

Settings.

Project-specific navigation may appear inside the selected project context. The global navigation must always make the active workspace visible.

6.2 Persistent identity strip

When a workspace is active, the UI **SHOULD** display a compact identity strip containing available information such as:

Workspace name.

Current project.

Current branch.

Git commit email.

GitHub account or organisation context.

Runtime version.

Selected cloud project when a supported service is active.

The strip **MUST** never display secrets.

6.3 Workspace switcher

The switcher **MUST** show:

Workspace name and icon.

Number of projects.

Running terminals or processes.

Attention indicators, such as uncommitted work, failed checks, or active installations.

Switching a workspace changes the visible project list, activity, terminals, GitHub content, environment status, and automation. It does not terminate processes unless the user chooses to stop them.

6.4 All-workspaces view

An optional All Workspaces overview MAY show high-level status across contexts. It MUST NOT combine credentials, execute commands, or blur which workspace owns an action. Any action initiated there must first resolve to one workspace.

7 Core user journeys

7.1 Journey A: First launch on a new computer

User launches Space.

Space explains that it can prepare the computer.

Space performs a non-mutating scan.

Space displays only missing or unhealthy essentials.

User reviews the plan and continues.

The operating system requests elevation only for the specific installer when needed.

Space installs tools through trusted package managers or official installers.

Space verifies versions and executable health.

Space writes receipts.

User names the first workspace.

User creates, adds, or clones a project.

Failure at any step must leave a recoverable state and a clear retry path. Completed steps must not repeat unnecessarily.

7.2 Journey B: Create a workspace

User selects New Workspace.

User enters a name.

User may optionally choose an icon and default project directory.

Space validates uniqueness of the internal identifier, not necessarily display-name uniqueness.

Space creates an empty workspace record and configuration directories.

Space opens the workspace home.

No service authentication is required at creation.

7.3 Journey C: Add an existing Git project

User selects a folder.

Space resolves the canonical path and checks permissions.

Space detects the repository root and worktree state.

Space detects remotes, branches, tags, status, project type, runtime requirements, and scripts.

Space checks whether the folder is already assigned to another workspace.

Space displays an import summary.

User confirms the workspace association.

Space indexes the repository and begins watching relevant files.

Space MUST NOT run dependency installation or arbitrary project scripts during detection.

7.4 Journey D: Add a folder without Git

User selects a folder.

Space detects that no repository exists.

Space detects project type and sensitive files.

Space proposes Git initialisation and an appropriate .gitignore.

User reviews files that will be included and ignored.

Space runs `git init -b main`, using a compatible fallback if the installed Git does not support that form.

Space creates or updates .gitignore without overwriting unrelated existing rules.

Space stages the approved files.

Space creates the first commit after confirming author identity.

Space offers to publish the same project to GitHub.

Space **MUST** block the first commit if likely secrets are included, unless the user explicitly resolves each warning.

7.5 Journey E: Publish a project to GitHub

Space verifies a clean or understood Git state.

Space verifies the workspace GitHub connection.

If unauthenticated, it launches the normal `gh` authentication flow in a PTY or browser-assisted flow.

Space lists available owners when possible.

User selects owner, repository name, description, visibility, and remote name.

Space checks whether the target repository exists.

Space shows the exact planned result.

Space runs the equivalent of `gh repo create OWNER/NAME --source=. --remote=origin --push` using structured arguments.

Space verifies the remote URL, upstream branch, and pushed commit.

Space stores the GitHub repository mapping and writes a receipt.

If repository creation succeeds but push fails, the UI must show that partial state and offer a safe retry rather than creating a duplicate repository.

7.6 Journey F: Clone from GitHub

User chooses Clone from GitHub.

Space checks workspace GitHub authentication.

Space lists or searches repositories accessible to the workspace account.

User chooses repository and destination.

Space checks destination collision and disk permissions.

Space clones using `gh repo clone` or `Git` with the workspace credential context.

Space detects project requirements.

Space asks whether to trust the project before installing dependencies or running scripts.

Space adds the project and opens it.

7.7 Journey G: Switch workspace

User selects another workspace.

UI immediately changes active context.

Space restores that workspace's projects, terminals, GitHub data, activity, and environment summary.

Existing background processes remain bound to their original workspace.

Any new operation uses the newly active workspace.

If the user attempts an action on a project owned by a different workspace, Space blocks or requests an explicit reassignment.

7.8 Journey H: Use a service for the first time

User runs a service command or opens a supported service panel.

The adapter detects installation and authentication state.

If the CLI is missing, Space offers to install it.

If authentication is missing, the CLI's normal authentication flow starts in the workspace context.

Space verifies the resulting account/profile.

The service appears in the workspace Environment view.

7.9 Journey I: Standard Git and GitHub workflow

User opens project changes.

Space reads real staged and unstaged changes.

The intent layer groups related changes without hiding file-level evidence.

User selects files or hunks.

Space stages exact patches.

User creates a checkpoint/commit.

Space offers branch creation, push, or pull request.

GitHub view follows checks, reviews, merge state, Actions, deployment, and release.

8 Onboarding and computer bootstrap specification

Minimal Onboarding Flow

One clear action at a time; every system change remains visible and recoverable.



8.1 ONB-001: First-run state

Space MUST persist a bootstrap state machine with these statuses:

not_started

scanning

plan_ready

installing

verification_required

partially_complete

complete

blocked

The state machine must be resumable after application restart, system restart, lost network, installer cancellation, or an individual step failure.

8.2 ONB-002: Non-mutating scan

The scan MUST collect:

Operating-system version and architecture.

Available disk space.

Shell availability.

Package-manager availability.

Git executable and version.

GitHub CLI executable and version.

Volta executable and version.

Node executable and version.

npm executable and version.

Supported editors.

Existing Space shell integration.

Whether the user can write to the selected default project location.

The scan MUST NOT modify PATH, install software, authenticate services, or create repositories.

8.3 ONB-003: Bootstrap manifest

The installer plan MUST be generated from a versioned manifest. Each tool entry must contain:

```
interface ToolManifestEntry {
  id: string;
  displayName: string;
  required: boolean;
  supportedPlatforms: Array<'darwin' | 'win32'>;
  architectures: string[];
  detection: DetectionRule[];
  installStrategies: InstallStrategy[];
  verify: VerificationRule[];
  minimumVersion?: string;
  recommendedVersionPolicy: 'latest-supported' | 'lts' | 'pinned';
  uninstallGuidance?: string;
}
```

The manifest MUST be signed or packaged with the application release. Remote manifest updates MAY be supported later but must be signature-verified.

8.4 ONB-004: macOS bootstrap

The supported macOS path MUST:

Check the supported operating-system minimum.

Check Apple Command Line Tools availability.

Detect Homebrew in standard Apple Silicon and Intel prefixes and through command -v.

Install Homebrew only after showing the official installer source and required changes.

Install missing essentials through Homebrew where appropriate.

Install Volta through an approved official strategy.

Install the current supported Node LTS through Volta.

Verify each executable using a direct process spawn.

Avoid editing shell files unless necessary and explicitly shown.

Homebrew is machine-scoped. Its installation may be interactive and may require the user to approve commands. Space MUST not hide password prompts or simulate system authentication dialogs.

8.5 ONB-005: Windows bootstrap

The supported Windows path MUST:

Check the supported Windows build and architecture.

Detect WinGet/App Installer.

Explain remediation when WinGet is unavailable.

Install Git for Windows, GitHub CLI, and Volta through WinGet or official signed installers.

Use PowerShell as the default shell integration, with optional Command Prompt or Git Bash support later.

Install the current supported Node LTS through Volta.

Verify ConPTY capability for integrated terminals.

handle User Account Control prompts through the normal Windows installer flow.

Space MUST NOT run the entire application as administrator. Only the specific installer process may request elevation.

8.6 ONB-006: Installation progress

Each installation step must expose:

Pending, running, successful, skipped, failed, cancelled, or needs-restart state.

Human-readable explanation.

Technical command or installer strategy.

Start and end time.

Redacted output.

Exit code.

Retry eligibility.

Whether the step changed machine state.

8.7 ONB-007: Verification

A tool is not considered installed merely because an installer returned zero. Space MUST verify:

Executable resolution from a newly constructed environment.

Version output.

Basic functional command where safe.

Architecture compatibility.

Expected path.

8.8 ONB-008: Receipts and recovery

For every machine change, Space MUST create an operation receipt. The receipt must distinguish:

Installed by Space.

Already present.

Updated by Space.

User-cancelled.

Partially installed.

Requires manual remediation.

Space SHOULD provide uninstall guidance but MUST NOT promise automatic rollback for package-manager operations that cannot be reliably reversed.

8.9 Default essential set

The first-run default set is intentionally small:

Git.

GitHub CLI.

Volta.

Current supported Node LTS through Volta.

npm, supplied with Node.

Space terminal integration.

Homebrew or WinGet is infrastructure for installing these tools, not a workspace dependency.

The following MUST remain optional or project-triggered:

pnpm and Yarn.

Python.

Docker.

WSL.

Java.

Android tooling.

Cloud CLIs.

Firebase CLI.

Vercel CLI.

Supabase CLI.

Database servers.

9 Workspace management requirements

9.1 WS-001: Create workspace

Required inputs:

Display name.

Optional inputs:

Icon or colour token.

Default project directory.

Initial Git author identity.

Space generates an immutable workspace UUID. Renaming changes only the display name and safe directory alias, not the UUID or credential references.

9.2 WS-002: Workspace storage

Workspace metadata **MUST** be stored in the local database. Workspace configuration roots **MUST** use an application-managed data directory appropriate to the operating system.

Suggested logical structure:

```
SpaceData/
  database/
    workspaces/
      <workspace-id>/
        git/
        github/
        ssh/
        services/
        environment/
        terminals/
        logs/
        cache/
```

Project source folders may remain anywhere the user chooses. Space stores canonical references; it does not move a project unless explicitly asked.

9.3 WS-003: Delete workspace

Deletion must show exactly what will be deleted:

Space metadata.

Workspace configuration directories.

Terminal history and local activity.

Secure-secret references and optional secret entries.

Local project folders **MUST NOT** be deleted by default. Remote repositories **MUST** never be deleted as part of workspace deletion. The user may remove project registration separately.

9.4 WS-004: Export and import

V1 **SHOULD** allow export of non-secret workspace configuration:

Name and icon.

Project path list.

Tool preferences.

Non-secret environment keys.

Automation definitions.

Secrets, access tokens, raw credential files, and terminal history **MUST** be excluded by default.

9.5 WS-005: Project ownership

A project path is normally owned by one workspace. When the same path is added elsewhere, Space must show:

Existing workspace owner.

Identity and credential mismatch risk.

Options to open in the existing workspace, move ownership, or deliberately share.

Shared ownership is an advanced option and must display a persistent warning because the same repository can then be acted on with different identities.

9.6 WS-006: Workspace switch atomicity

The UI context switch must be atomic from the user's perspective. Operations already running continue under the environment snapshot with which they started. New operations bind to the new active workspace.

The application **MUST NOT** mutate a running process's environment after launch.

10 Project management requirements

10.1 PRJ-001: Project registration

A project record must contain:

UUID.

Workspace UUID.

Display name.

Canonical path.

Filesystem identity where available.

Repository root, if any.

Project type detections.

Trust state.

Remote mapping.

Last-opened time.

Health state.

10.2 PRJ-002: Detection

Detection MUST be read-only and inspect known indicators such as:

.git file or directory.

package.json and lockfiles.

Volta metadata.

framework configuration files.

Python, Rust, Java, .NET, and other common manifests when support is added.

Environment-file names without reading secret values into telemetry.

Common generated directories.

The detector should produce confidence-scored facts rather than one irreversible label.

10.3 PRJ-003: Trust model

Newly cloned, downloaded, or unfamiliar projects start as untrusted.

While untrusted, Space MAY read ordinary source files and Git metadata but MUST NOT automatically:

Run package installation.

Run lifecycle scripts.

Execute project binaries.

Source shell files.

Load arbitrary Electron content from the project.

Inject workspace secrets.

The trust prompt must explain that dependency installation and project scripts can execute code.

Trust choices:

Allow once.

Trust this project.

Keep untrusted.

10.4 PRJ-004: New project creation

A project template provider must define:

Supported platform.

Required CLI and runtime.

Non-interactive creation command.

User-editable options.

Expected files.

Verification command.

Development command.

Default ignore rules.

The user may begin with a natural-language description, but the UI **MUST** show the concrete framework, language, package manager, and creation command before execution.

10.5 PRJ-005: Dependency installation

Space must detect the lockfile and prefer the matching package manager. It **MUST** not silently convert package managers or regenerate lockfiles.

Examples:

pnpm-lock.yaml -> pnpm.

yarn.lock -> Yarn.

package-lock.json -> npm.

If multiple incompatible lockfiles exist, Space must ask rather than guess.

10.6 PRJ-006: Development server

When a known development script is available, Space may offer Run. It must:

Launch through the Workspace Process Runner.

Attach output to a durable process record.

Detect the listening URL from structured configuration or output.

Show stop/restart actions.

Preserve workspace ownership after switching context.

Avoid exposing a development server outside localhost unless the user intentionally configures it.

11 Git engine specification

11.1 Mandatory decision

The system Git executable is the source of truth in V1. Do not introduce libgit2 as a second repository engine. A future ADR may add a library for a narrowly justified feature, but it must not create conflicting state.

11.2 Git executable discovery

The Git service must resolve Git from the machine tool registry and verify it. It must not rely solely on the renderer's inherited PATH.

11.3 Structured Git commands

Use machine-readable output wherever Git provides it. Commands must be constructed as executable plus argument array, not concatenated shell strings.

11.3.1 Repository root and state

```
git rev-parse --show-toplevel
git rev-parse --git-dir
git rev-parse --is-inside-work-tree
git status --porcelain=v2 -z --branch
```

11.3.2 Refs

```
git for-each-ref --format=<NUL-safe structured format> refs/heads refs/remotes refs/tags
```

11.3.3 History

```
git log --all --topo-order --date-order --parents --decorate=full --format=<record format>
```

Use explicit record and field separators that cannot collide with normal text. The parser must be covered by fixtures containing Unicode, newlines, unusual author names, merge commits, tags, detached HEAD, shallow clones, replace refs, and large histories.

11.3.4 Diffs

```
git diff --no-ext-diff --binary-safe-options
git diff --cached --no-ext-diff
git diff --raw -z
git diff --numstat -z
```

External diff tools and textconv must not be invoked by default because they can execute user configuration. Any use requires an explicit trust and security review.

11.4 GIT-001: Status model

The status parser must represent:

Branch and upstream.

Ahead/behind counts.

Initial repository with no commits.

Staged and unstaged modifications.

Added, deleted, renamed, copied, type-changed, and unmerged entries.

Untracked and ignored files.

Submodule state.

Ongoing merge, rebase, cherry-pick, revert, bisect, and sequencer states.

11.5 GIT-002: Git initialisation

Before git init, Space must:

Check whether the selected folder is already inside a parent repository.

Warn if initialising would create a nested repository.

Determine the requested default branch.

Confirm write access.

Inspect likely secrets and generated directories.

11.6 GIT-003: Identity

Before the first commit in a workspace or after an identity mismatch, Space must resolve:

Author name.

Author email.

Signing policy.

Signing key availability when required.

A repository-local identity may override the workspace identity only after an explicit user action.

11.7 GIT-004: Staging

Space must support:

Stage/unstage file.

Stage/unstage selected hunk.

Stage/unstage selected lines where technically safe.

Discard selected unstaged changes with confirmation and recovery guidance.

Patch application must use generated patch files or stdin, never fragile shell escaping.

11.8 GIT-005: Commit/checkpoint

A commit operation must:

Refresh status immediately before execution.

Verify the intended index state.

Run optional configured pre-commit hooks in a visible process.

Display hook failures accurately.

Create the real Git commit.

Verify the resulting SHA.

Record activity and receipt.

Space MUST NOT bypass hooks unless the user chooses an advanced action and is warned.

11.9 GIT-006: Branch operations

Support:

Create from selected commit.

Switch/checkout.

Rename.

Delete with merged-state checks.

Publish/upstream association.

Compare.

Destructive branch deletion and force operations require explicit confirmation.

11.10 GIT-007: Fetch, pull, and push

Fetch may be a normal remote action with clear status.

Pull must make merge versus rebase behaviour explicit and respect repository configuration.

Push must show workspace identity, remote, branch, and whether it is a force operation.

Force push must use --force-with-lease by default and require confirmation.

Raw --force must be hidden behind an advanced confirmation and policy gate.

11.11 GIT-008: Conflicts

Conflict state must be represented explicitly. Space must not imply a merge or rebase is complete until Git confirms it. The UI should offer file-level conflict navigation and safe continue/abort commands.

11.12 GIT-009: Repository watcher

Use a filesystem watcher to trigger debounced refresh, but never assume a watch event fully describes repository state. Git status remains authoritative.

Watch at minimum:

Worktree paths, with ignore rules for large generated directories.

.git/HEAD.

.git/index.

.git/refs and packed refs.

operation-state files.

11.13 Git operation queue

Mutating Git operations for the same repository must be serialised. Read operations may be concurrent when they cannot conflict, but status refreshes must be coalesced. Use per-repository locks, not one global lock.

12 Git history and graph specification

12.1 HIS-001: Full workspace use

The History view must use the available main workspace, not a narrow card. It is a primary product surface.

12.2 HIS-002: Required columns and objects

Graph lanes.

Working-tree node when applicable.

HEAD marker.

Local branches.

Remote branches.

Tags.

Commit subject.

Author.

Short SHA.

Relative time and exact time on demand.

Signature state where available.

Pull-request and check status when mapped.

12.3 HIS-003: Graph data

Each commit model must include:

```
interface CommitNode {
  sha: string;
  parents: string[];
  subject: string;
  body?: string;
  authorName: string;
  authorEmail?: string;
  authoredAt: number;
  committedAt: number;
  refs: RepositoryRef[];
  signature?: SignatureState;
}
```

The renderer MUST derive topology from commit and parent SHAs. It MUST NOT parse the ASCII art produced by `git log --graph`.

12.4 HIS-004: Lane layout

The lane algorithm must:

Process commits in the displayed topological order.

Maintain stable active lanes.

Keep the first parent on the current lane where possible.

Allocate lanes for additional parents.

Represent branch divergence and merge convergence.

Preserve lane stability when another page of history loads.

Produce deterministic output for the same commit sequence.

The algorithm requires unit fixtures for linear history, simple branch, octopus merge, repeated merges, criss-cross history, hidden refs, shallow boundaries, and filtered history.

12.5 HIS-005: Rendering performance

Commit rows must be virtualised.

Graph drawing should use Canvas or another measured efficient layer.

Text and interactive labels should remain accessible DOM elements.

The graph must remain responsive with at least 100,000 indexed commits while rendering only the visible window.

Initial visible history should appear before full repository indexing completes.

12.6 HIS-006: Selection inspector

Selecting a commit opens an inspector showing:

Full message.

Author and committer.

Exact dates.

Parents and children where known.

Refs.

Changed files and statistics.

Patch on demand.

GitHub association.

Checks and signatures.

Actions such as create branch, tag, compare, or copy SHA.

13 Changes and intent layer

13.1 CHG-001: Technical truth first

The source data is the real Git diff. The intent layer is an interpretation, not a replacement.

Every intent group must preserve traceability to exact:

File paths.

Hunks.

Line ranges.

Staged or unstaged state.

13.2 CHG-002: Intent groups

An intent group contains:

```
interface ChangeIntent {
  id: string;
  title: string;
  explanation: string;
  userVisibleOutcome?: string;
  confidence: number;
  evidence: DiffSelection[];
  included: boolean;
  generatedBy: 'rule' | 'model' | 'user';
}
```

Low-confidence interpretations must be labelled. The user can rename, merge, split, include, or exclude groups.

13.3 CHG-003: Model privacy

If a remote model is used:

The user must know which files or diff fragments are sent.

Secrets and known sensitive values must be redacted.

Binary files must not be sent by default.

The model must receive the minimum evidence required.

Raw credentials, environment files, and key material must never be sent.

The operation must be cancellable and auditable.

A local or offline grouping fallback must exist, even if less sophisticated.

13.4 CHG-004: Commit composition

The user can compose a commit from selected intent groups, files, hunks, or lines. Space must refresh the diff before applying a generated patch and detect stale selections.

14 GitHub integration specification

14.1 Mandatory decision

GitHub CLI is the primary GitHub integration in V1. Space owns the workspace abstraction; the implementation MUST NOT depend on a hypothetical GitHub CLI workspace feature.

14.2 GitHub capabilities

The GitHub area contains:

Overview.

Pull requests.

Issues.

Checks.

Actions.

Deployments where available.

Releases.

Automation.

14.3 GH-001: Authentication

Space must support normal gh auth login flows in an interactive PTY. It must show the resulting hostname and account but never expose the token.

The GitHub adapter must report:

CLI installed state.

Authentication state by host.

Active workspace account.

Available organisations where obtainable.

Git protocol.

Token-source strategy.

14.4 GH-002: Credential setup for Git

When HTTPS is selected, Space may use gh auth setup-git inside the workspace Git configuration strategy. The outcome must be verified with a non-destructive remote authentication check.

14.5 GH-003: Repository creation

The create/publish form requires:

Owner.

Name.

Visibility.

Optional description.

Source folder.

Remote name, default origin.

Push choice.

The operation must be idempotent-aware. If the remote repository already exists, Space must offer connect, choose another name, or cancel; it must not silently overwrite.

14.6 GH-004: Pull requests

Support:

List and filter.

Open detail.

Create from current branch.

Generate/edit title and body.

Select base and head.

Add reviewers, assignees, labels, and linked issues where permissions allow.

View files, commits, timeline, review state, mergeability, and checks.

Checkout a PR.

Merge only with explicit confirmation and policy checks.

14.7 GH-005: Checks

The PR detail must display:

Name.

State.

Workflow/job association.

Start and finish time.

Details URL.

Required-check status.

Space should use structured JSON output and may watch check status. Polling must use backoff and stop when the view is closed or the operation is complete.

14.8 GH-006: Actions

Support:

List workflows.

View recent runs.

Trigger a workflow_dispatch workflow with branch/tag and input values.

Watch a run.

View jobs and logs.

Download artifacts when selected.

Cancel.

Rerun failed jobs or the complete run.

Space must not invent inputs; it should read workflow metadata or ask for explicit values.

14.9 GH-007: Issues

Support list, search, detail, create, edit, assign, label, comment, close/reopen, and start-work flow.

Start work should be a guided operation that:

Refreshes the base branch.

Creates a branch using a safe slug.

Records the issue as the session goal.

Opens the relevant project.

Links the future PR to the issue.

14.10 GH-008: Releases

Support:

Compare changes since a selected tag.

Draft version suggestion, subject to user confirmation.

Generate editable release notes.

Create and push tag.

Trigger release workflow.

Upload selected artifacts.

Publish or save as draft/prerelease.

Publishing is a high-impact remote action and requires confirmation.

14.11 GH-009: Offline and degraded behaviour

When GitHub is unavailable:

Local Git remains fully available.

Cached GitHub data is labelled with last refresh time.

Remote actions are disabled with a clear reason.

Queuing an automatic remote mutation for later is not allowed in V1 unless the user explicitly creates a draft operation and reviews it when connectivity returns.

15 Terminal specification

15.1 TERM-001: Real terminal

Use xterm.js for presentation and node-pty for a real shell process. The terminal is not a simulated command console.

Default shells:

macOS: user's supported login shell, normally zsh.

Windows: PowerShell, with optional Git Bash when installed.

15.2 TERM-002: Workspace binding

A terminal session is permanently bound to the workspace and project context at creation. Switching the active workspace hides but does not rebind the session.

15.3 TERM-003: Environment

The shell starts with the environment produced by the Workspace Process Runner. Space must avoid sourcing untrusted project files automatically.

15.4 TERM-004: Lifecycle

Track:

PTY process identifier.

Workspace and project.

Shell.

Current directory.

Start time.

Exit status.

Running foreground operation where detectable.

Last output timestamp.

On application restart, terminated sessions may be represented as history; live PTY processes cannot be assumed recoverable unless a separate durable process host is intentionally implemented later.

15.5 TERM-005: Output handling

Use backpressure to prevent renderer overload.

Do not persist raw terminal output by default indefinitely.

Redact known injected secrets from operation logs.

Interactive terminal history remains subject to the user's shell behaviour; Space should not scrape secrets from it.

15.6 TERM-006: Links and browser opening

Detected URLs must be opened through a validated external-navigation service. The renderer must not navigate to arbitrary local or remote content with Node privileges.

16 Environment and service adapters

16.1 ENV-001: Tool registry

The Environment view must report:

Tool name.

Installed state.

Version.

Path.

Install source.

Scope.

Health.

Update availability when safely known.

Projects that require it.

16.2 ENV-002: Version policy

Space should pin and test exact versions in its release pipeline, but user machines may contain newer compatible versions. Detection must use semantic version ranges and capability checks, not only equality.

Updates must be explicit. Space must not update Node or a critical CLI while a dependent process is running without warning.

16.3 ENV-003: Volta and Node

Volta is machine-scoped. Space uses it to install supported Node versions and to respect project pins.

Rules:

Existing project pins win.

If a project has no pin, workspace default policy may be proposed.

New projects should receive an explicit pin where supported.

Space must not silently edit an existing project's runtime metadata.

Node versions may share Volta's machine cache; identity and credentials remain workspace-scoped.

16.4 ENV-004: Service adapter contract

```
interface WorkspaceServiceAdapter {
  id: string;
  displayName: string;
  isolationTier: 1 | 2 | 3 | 4;
  detect(ctx: WorkspaceContext): Promise<ServiceDetection>;
  buildEnvironment(ctx: WorkspaceContext): Promise<Record<string, string>>;
  getAuthState(ctx: WorkspaceContext): Promise<AuthState>;
  beginAuthentication(ctx: WorkspaceContext): Promise<InteractiveOperation>;
  verifyAuthentication(ctx: WorkspaceContext): Promise<AuthState>;
  disconnect(ctx: WorkspaceContext): Promise<OperationResult>;
  redact(values: RedactionRegistry): Promise<void>;
}
```

Adapters must not receive renderer objects or unvalidated free-form commands.

16.5 Initial adapter roadmap

16.5.1 V1 required

Git.

GitHub CLI.

Volta/Node.

16.5.2 V1.1 recommended

Google Cloud CLI.

Vercel CLI.

Firebase CLI.

16.5.3 Later

AWS CLI.

Azure CLI.

Docker context.

Kubernetes context.

Supabase CLI.

npm registry authentication.

Each later adapter requires its own threat model and cross-platform tests.

17 Activity specification

17.1 ACT-001: Real event source

The activity system records real events from operations and observed repository state. It must not populate production with generated activity.

Event categories include:

Workspace created or switched.

Project added, created, cloned, or removed.

Tool installed or verified.

Terminal session.

Git initialised.

Commit/checkpoint.

Branch created or switched.

Fetch/pull/push.

Pull request opened, reviewed, merged, or closed.

Check or workflow outcome.

Release or deployment.

Automation execution.

17.2 ACT-002: 52-week grid

The activity grid must:

Render a complete 52-week period.

Use the available width responsively.

Show month and weekday alignment correctly.

Distinguish intensity by event count or weighted meaningful activity.

Provide accessible labels and day detail.

Avoid equating raw keystrokes or file saves with meaningful productivity.

17.3 ACT-003: Daily detail

Selecting a day shows events grouped by workspace and project with links to the relevant commit, PR, workflow, release, or receipt.

17.4 ACT-004: Privacy

Activity is local by default. Cross-device synchronisation is out of V1 scope unless separately consented and designed.

18 Automation specification

18.1 Automation model

An automation has:

Workspace owner.

Optional project scope.

Trigger.

Conditions.

Ordered actions.

Permission requirements.

Enabled state.

Last execution.

Execution history.

18.2 V1 local triggers

Project opened.

File/repository state changed.

Commit created.

Branch pushed.

Check failed while app is running.

Development process exited.

Scheduled time while app is running.

18.3 V1 actions

Run an approved project script.

Run tests.

Fetch repository.

Notify the user.

Prepare, but not silently publish, a PR description.

Trigger a selected GitHub workflow after explicit permission.

Open a receipt or relevant view.

18.4 Safety rules

Automations inherit workspace and project boundaries.

Remote mutations require an explicit standing permission scoped to the exact action.

Destructive operations cannot be fully automated in V1.

A user can disable all automations immediately.

Each run produces a receipt.

Failed automations use bounded retries and never loop indefinitely.

18.5 Later GitHub App phase

Background automation while Space is closed requires a Space GitHub App and backend. That phase may add:

Webhooks.

Organisation installation.

Custom check runs.

Scheduled cloud-side rules.

Notifications while desktop is offline.

The desktop MVP must not depend on this backend.

19 Agent and computer interaction specification

The agent is an orchestration layer over approved operations. It must not be given unrestricted access to the shell or credential store.

19.1 Typed action plan

The model produces a proposed plan using a constrained schema. Example:

```
interface AgentAction {
  id: string;
  type:
    | 'file.read'
    | 'file.modify'
    | 'project.installDependencies'
    | 'project.runScript'
    | 'git.stage'
    | 'git.commit'
    | 'git.push'
    | 'github.createPullRequest';
  workspaceId: string;
  projectId: string;
  parameters: unknown;
  explanation: string;
  risk: 'observe' | 'local-reversible' | 'remote' | 'destructive';
}
```

Every action is validated and translated by trusted application code.

19.2 Permission levels

19.2.1 Observe

Examples: read approved project files, inspect status, read history, inspect package manifest.

19.2.2 Local reversible

Examples: edit files, create branch, install dependencies in a trusted project, run tests, stage, commit.

19.2.3 Remote

Examples: push, open PR, comment, trigger workflow.

19.2.4 Destructive or high impact

Examples: force push, hard reset, delete branch, publish release, production deployment, delete files outside recoverable scope.

Destructive actions require confirmation every time. Remote standing permissions must be narrow and revocable.

19.3 Agent data controls

Secret values are never included in model prompts.

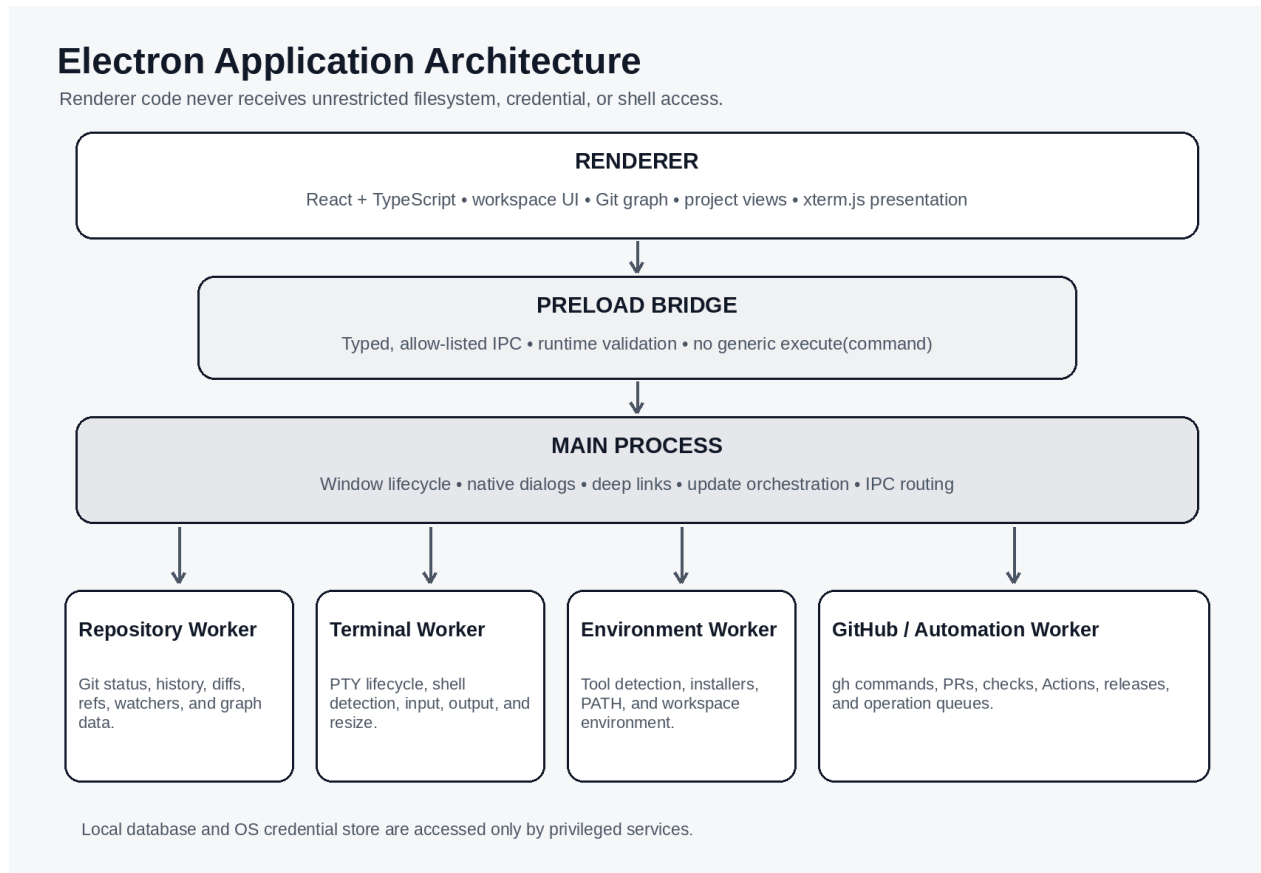
The model receives file content only when necessary and approved by policy.

Binary files and large generated folders are excluded.

External model calls are logged as metadata, not raw sensitive content.

Model output is untrusted input and must pass schema validation.

20 Electron architecture



20.1 Technology baseline

The implementation baseline is:

Electron on a currently supported stable major, pinned to an exact tested version.

TypeScript with strict mode.

React for the renderer.

Vite-based renderer build or the supported Electron Forge integration chosen in ADR-001.

Electron Forge for packaging and makers.

xterm.js for terminal presentation.

node-pty for PTY processes.

SQLite for local metadata and operation history.

Zod or an equivalent runtime schema validator at every IPC boundary.

A structured logging library with redaction support.

The team must update to the latest supported patch within the selected Electron major before each release and rerun the full native-module matrix.

20.2 Process responsibilities

20.2.1 Renderer

May:

Render UI.

Hold view state.

Request typed operations.

Display streamed progress.

Must not:

Import Node core modules.

Access filesystem directly.

Spawn processes.

Read credential storage.

Load arbitrary remote pages with privileged APIs.

Receive a generic execute method.

20.2.2 Preload

Exposes a minimal namespaced API.

Uses contextBridge.

Validates shape before sending.

Does not expose ipcRenderer directly.

Does not expose broad file or shell primitives.

20.2.3 Main process

Creates windows and web contents.

Applies security preferences.

Validates navigation and permission requests.

Routes IPC to services.

Coordinates application updates and deep links.

Does not perform long Git/indexing tasks on the UI thread.

20.2.4 Utility/worker processes

Separate logical services for:

Repository indexing and Git.

Terminal/PTY hosting.

Environment/tool detection and installers.

GitHub and automation.

Optional model orchestration.

Crashes in one worker should not corrupt other service state. The main process restarts workers with bounded policy and surfaces failures.

20.3 Window security defaults

Every production window must use:

```
const webPreferences = {
  nodeIntegration: false,
  contextIsolation: true,
  sandbox: true,
  preload: trustedPreloadPath,
  webSecurity: true,
};
```

Additional mandatory controls:

Strict Content Security Policy.

No allowRunningInsecureContent.

Deny unexpected permission requests.

Validate IPC sender frame and origin.

Restrict navigation.

Open external links through the system browser only after validation.

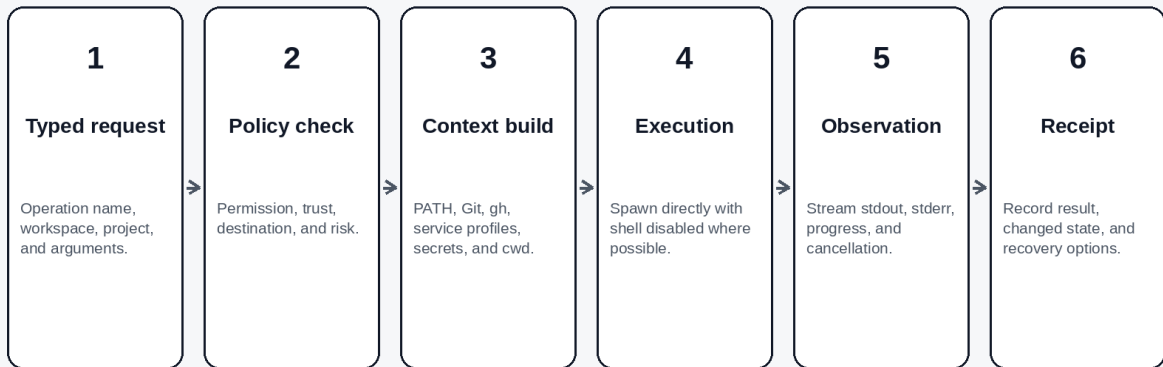
No remote code execution or remote module.

Disable unnecessary Electron fuses and capabilities in packaged builds.

21 Workspace Process Runner

Workspace Command Execution Pipeline

Every command, terminal, automation, and editor launch uses the same context builder.



Non-negotiable: the renderer may request approved operations; it may never submit arbitrary shell text.

21.1 Central responsibility

Every command, terminal, Git operation, GitHub operation, installer, project script, automation, and editor launch initiated by Space must flow through the Workspace Process Runner or an explicitly documented equivalent.

21.2 Structured request

```

interface ProcessRequest {
  operationId: string;
  workspaceId: string;
  projectId?: string;
  executableId: KnownExecutable;
  args: string[];
  cwd?: string;
  mode: 'captured' | 'pty' | 'detached';
  envPolicy: EnvironmentPolicy;
  timeoutMs?: number;
  stdin?: 'none' | 'pipe' | 'pty';
  risk: RiskLevel;
}
  
```

The renderer **MUST NOT** provide an arbitrary executable path for protected operations. Trusted backend code resolves KnownExecutable through the tool registry.

21.3 Command construction

Use `spawn(executable, args, { shell: false })` for captured non-interactive operations.

Use PTY mode for interactive shells and authentication.

Never concatenate user fields into a shell command string.

Validate repository names, branch names, remote names, paths, and enum-like values before command construction.

Use stdin or temporary files for commit messages, PR bodies, and patches when appropriate.

Create temporary files with restrictive permissions and delete them after use.

21.4 Cancellation

Operations must support cancellation where the underlying process can be safely terminated. Cancellation state is not the same as failure. The receipt must record whether partial side effects may remain.

21.5 Timeouts

Short read commands need bounded timeouts. Long interactive installs, tests, and workflows use explicit user-visible duration rather than an arbitrary hidden timeout.

21.6 Output and redaction

The runner must:

Stream stdout/stderr with ordering metadata.

Apply secret redaction before persistence or renderer delivery where possible.

Limit retained output size.

Store a final digest and relevant tail for large logs.

Preserve raw output only in an explicitly protected debug mode.

Never log complete process environments.

21.7 Operation state machine

```
created -> awaiting_permission -> queued -> running  
running -> succeeded | failed | cancelled | partially_succeeded | needs_user_input
```

A partially successful operation requires a specific remediation path.

22 IPC contract

22.1 Principles

Namespace by domain: workspace, project, git, github, terminal, environment, operation.

Request/response schemas are shared TypeScript types plus runtime validation.

Every request includes an operation correlation ID.

Every handler validates the sender.

Streaming uses dedicated channels scoped to the operation ID.

No channel accepts arbitrary JavaScript, command text, SQL, or filesystem glob from the renderer without domain validation.

22.2 Example preload API

```
interface SpaceAPI {
  workspace: {
    list(): Promise<WorkspaceSummary[]>;
    create(input: CreateWorkspaceInput): Promise<Workspace>;
    activate(workspaceId: string): Promise<void>;
  };
  project: {
    inspectFolder(input: InspectFolderInput): Promise<ProjectInspection>;
    add(input: AddProjectInput): Promise<Project>;
    create(input: CreateProjectInput): Promise<OperationHandle>;
  };
  git: {
    status(projectId: string): Promise<GitStatus>;
    history(input: HistoryQuery): Promise<HistoryPage>;
    initialise(input: InitialiseGitInput): Promise<OperationHandle>;
    commit(input: CommitInput): Promise<OperationHandle>;
  };
  operation: {
    subscribe(id: string, listener: (event: OperationEvent) => void): Unsubscribe;
    cancel(id: string): Promise<void>;
  };
}
```

There must be no API resembling:

```
runCommand(command: string): Promise<string>
```

23 Local data architecture

23.1 Database choice

Use SQLite behind a repository/data-access abstraction. Native modules must run in worker processes and be rebuilt and tested for each Electron platform/architecture.

23.2 Core entities

23.2.1 Workspace

id
name
icon_token
default_project_directory
active
created_at
updated_at

23.2.2 Project

id
workspace_id
name
canonical_path
filesystem_identity
repository_root
trust_state
detected_type_json
remote_json
last_opened_at

23.2.3 TerminalSession

id
workspace_id
project_id
shell
cwd
state
started_at
ended_at
exit_code

23.2.4 ToolInstallation

tool_id
scope

version
path
source
health
detected_at

23.2.5 ServiceConnection

id
workspace_id
adapter_id
host
account_label
profile_label
secret_ref_id
state
last_verified_at

23.2.6 SecretRef

id
workspace_id
provider
service_name
account_key
created_at

The secret value itself is not stored in SQLite.

23.2.7 Operation

id
workspace_id
project_id
type
risk
state
human_summary
started_at
ended_at
exit_code
partial_state_json

23.2.8 OperationStep

id
operation_id
sequence
type

state
command_display_redacted
result_summary

23.2.9 ActivityEvent

id
workspace_id
project_id
event_type
occurred_at
subject_ref
summary
weight
metadata_json

23.2.10 Automation

id
workspace_id
project_id
name
trigger_json
conditions_json
actions_json
permission_json
enabled

23.3 Migrations

All schema changes require versioned, tested migrations.

Migrations run before services start using the database.

Backups are created before destructive migrations.

A failed migration prevents mutation operations and shows recovery instructions.

Downgrade compatibility is not assumed; release notes must state database changes.

24 Credential and secret storage

24.1 Requirements

Use the operating system credential store through a reviewed abstraction.

Electron safeStorage may encrypt local small values but must be evaluated carefully by platform; use established keychain/credential APIs where stronger semantics are required.

SQLite stores only opaque secret references and metadata.

Secret values exist in memory for the shortest possible time.

Secrets are injected only into approved child processes.

Secrets are never returned to the renderer.

Clipboard copying requires an explicit user action and optional timed clearing.

24.2 Redaction registry

When a secret is loaded, its exact value and safe variants must be registered with the operation's redactor. Redaction is defence in depth; the design must still avoid logging secrets in the first place.

24.3 Workspace deletion

The user must choose whether to remove associated credential-store entries. Failure to remove one entry must be reported without losing the workspace-deletion result.

25 Security and threat model

25.1 Protected assets

GitHub and cloud tokens.
SSH private keys.
Signing keys.
Environment secrets.
Source code.
User files outside registered projects.
Remote repositories and deployment environments.
Git history integrity.
Package-manager and system integrity.

25.2 Threat actors and failure sources

Malicious or compromised project repository.
Malicious dependency or lifecycle script.
Prompt-injected model output.
Renderer cross-site scripting.
Compromised external website opened inside the app.
Command injection through branch, path, repository, or issue text.
Credential leakage through logs or process environment.
Wrong-workspace user error.
Untrusted installer or supply-chain compromise.
Native-module crash or memory issue.

25.3 Mandatory mitigations

25.3.1 Renderer compromise

Context isolation and sandbox enabled.
Strict CSP.
No Node integration.
Minimal preload API.
IPC sender validation.
No privileged remote content.
External navigation validation.

25.3.2 Command injection

shell:false for structured operations.
Arguments as arrays.
Domain validation.
Temporary files/stdin for rich text.
Never execute branch names or repository content as shell code.

25.3.3 Malicious project

Trust gate before scripts or installs.

Do not source project shell files automatically.

Do not render project HTML in a privileged webview.

Do not grant workspace secrets by default.

Show executable scripts before first run.

25.3.4 Wrong workspace/account

Before remote or deployment actions, show:

Workspace.

Project.

Remote owner/repository.

GitHub account.

Branch.

Cloud project/environment where relevant.

Block known mismatches and require explicit override.

25.3.5 Installer safety

Trusted package managers or official signed installers only.

HTTPS source verification.

Package identity allow-list.

Signed Space manifest.

Show elevation reason.

Verify result.

25.3.6 Model safety

Model output is data, never executable authority.

Typed plans only.

Permission gates.

Secret stripping.

Human confirmation for high-impact actions.

25.4 Workspace limitation disclosure

Workspace separation protects against accidental cross-context use. It does not contain malicious software. A future container/VM mode may provide stronger isolation and must be described separately.

26 Error handling and recovery

26.1 Error design

Every error should answer:

What failed?

What completed before the failure?

Was any state changed?

What can the user do now?

Where can technical details be viewed?

26.2 Error categories

Validation.

Permission.

Authentication.

Missing tool.

Unsupported version.

Network.

Command failure.

Conflict.

Partial remote operation.

Native worker crash.

Database/migration.

Security policy block.

26.3 Partial operation examples

26.3.1 GitHub repository created, push failed

Record remote repository URL, add or verify origin, and offer Retry Push. Do not rerun repository creation blindly.

26.3.2 Tool installer completed, PATH not visible

Rebuild process environment and verify. If restart is required, mark needs_restart rather than failed.

26.3.3 Commit succeeded, activity write failed

The Git commit is authoritative. Repair activity asynchronously and do not claim commit failure.

26.3.4 PR created, reviewer assignment failed

Show the PR as created and list the failed secondary step.

27 Performance requirements

27.1 Startup

Show the application shell quickly from local state.

Do not block first paint on GitHub network calls or repository indexing.

Restore the active workspace and project from the local database.

27.2 Repository interaction

Status refresh should feel immediate for ordinary repositories.

Large repositories require cancellable background work and progressive results.

Diff loading is on demand and paginated/virtualised for large files.

Binary files are represented by metadata, not loaded as text.

27.3 UI

Long lists are virtualised.

Worker messages are batched.

Terminal output uses backpressure.

No synchronous filesystem or database access in the renderer.

No long-running work on the Electron main thread.

27.4 Resource limits

The team must define and test limits for:

Maximum retained operation output.

Activity retention.

GitHub cache size.

Number of simultaneous PTYs.

Concurrent repository reads.

Install/download cache.

28 Accessibility and interaction requirements

Full keyboard navigation for primary actions.

Visible focus states.

Screen-reader labels for graph nodes, activity cells, status icons, and controls.

Colour is not the only signal for branch, status, or activity intensity.

Respect operating-system reduced-motion settings.

Support high-contrast themes.

Terminal remains keyboard accessible.

Relative times expose exact accessible labels.

Error messages are announced and linked to the failed field or operation.

29 Privacy, analytics, and logging

29.1 Local by default

Project paths, Git history, terminal sessions, activity, and credentials remain local unless a specific remote operation requires them.

29.2 Telemetry

Telemetry must be opt-in or clearly disclosed according to product policy. It may include:

Anonymous feature usage.

Performance timings.

Crash category.

Installer outcome by tool and platform.

It must not include:

Source code.

Commit messages by default.

File paths without sanitisation.

Terminal text.

Tokens or environment values.

Repository private names unless explicitly consented.

29.3 Logs

Logs must use levels, correlation IDs, and redaction. Production logs need rotation and bounded retention. A support bundle must preview exactly what will be exported.

30 Cross-platform implementation details

30.1 macOS

Package format: signed and notarised application, distributed through an appropriate installer image or package.

Architectures: Apple Silicon and Intel where supported by the product decision.

Shell: zsh baseline.

Credential store: Keychain-backed abstraction.

File watching: account for case-sensitive and case-insensitive filesystems.

Quarantine and Gatekeeper behaviour must be tested.

Do not depend on Homebrew being installed.

30.2 Windows

Package format: signed installer.

Baseline should favour Windows 11 while documenting the exact supported Windows 10 build if included.

Shell: PowerShell baseline.

Terminal: ConPTY through node-pty.

Credential store: Windows Credential Manager or reviewed equivalent.

Handle drive letters, UNC paths, long paths, reserved names, and case-insensitive behaviour.

Test antivirus/Defender interactions with PTYs, installers, and development servers.

30.3 Path rules

Store canonical paths using platform-aware APIs.

Never concatenate paths manually.

Preserve display path separately from canonical path where useful.

Treat symlinks, junctions, worktrees, and network drives explicitly.

Validate that operations remain within approved project roots when expected.

31 Packaging, updates, and release engineering

31.1 Electron Forge

Use Electron Forge for packaging and makers unless ADR-001 selects a documented equivalent. The pipeline must support:

Platform packaging.

Native module rebuilding.

Code signing.

macOS notarisation.

Windows installer signing.

Release publishing.

Update artifacts.

31.2 Code signing

Unsigned public releases are not acceptable.

macOS: Developer ID signing and notarisation.

Windows: trusted code-signing certificate and timestamping.

Secrets for signing live in CI secret storage and are never committed.

31.3 Auto-update

Updates must be signed and verified.

The UI must not interrupt running installations, Git mutations, or unrecoverable operations.

Database migrations are considered before applying an update.

Allow defer/restart-later behaviour.

Maintain a documented rollback policy; app binary rollback may not imply database rollback.

31.4 Release channels

Recommended channels:

Internal development.

Alpha.

Beta.

Stable.

A workspace database created by a newer channel must not silently corrupt when opened by an older channel.

32 Recommended repository structure

```

space/
  apps/
    desktop/
      src/main/
      src/preload/
      src/renderer/
      forge.config.ts
  packages/
    contracts/           # IPC schemas and shared domain types
    domain/              # pure models and policies
    workspace-runner/    # environment construction and process execution
    git-engine/          # commands, parsers, queue, fixtures
    github-engine/       # gh adapter and GitHub models
    environment/         # tool registry, installers, service adapters
    terminal/            # PTY host and protocol
    activity/            # event model and aggregation
    automation/          # triggers, conditions, actions
    storage/             # SQLite repositories and migrations
    security/            # permission policy, redaction, sender validation
    ui/                  # shared renderer components
  tests/
    fixtures/
    integration/
    e2e/
  docs/
    adr/
    threat-model/
    runbooks/

```

Packages that execute native operations must not be imported into renderer bundles.

33 Coding standards

TypeScript strict mode, no unchecked any in boundary code.

Runtime validation for external process output and IPC input.

Exhaustive state handling for operations.

Pure parsers with fixtures.

Dependency injection for executable resolution and process spawning in tests.

No hidden command execution in React components.

No business logic in preload.

No direct SQLite access from renderer or React hooks.

Security-sensitive changes require reviewer approval from an assigned owner.

Every mutating command has a corresponding receipt and test.

Every user-visible async flow defines loading, empty, success, failure, partial, offline, and cancelled states.

34 Testing strategy

34.1 Unit tests

Required for:

Git porcelain parser.

Ref and history parser.

Lane-layout algorithm.

Environment precedence.

Adapter configuration construction.

Command argument validation.

Permission policy.

Redaction.

Project detection.

Operation state transitions.

Database migrations.

34.2 Integration tests

Run against temporary real repositories and stubbed isolated service configuration:

Initialise repository.

First commit.

Branch, merge, tag, detached HEAD.

Staging hunks.

Conflict states.

Multiple worktrees.

Publish-operation partial failures using controlled test doubles.

Multiple workspace Git identities.

Multiple GitHub auth contexts after P0 spike.

Terminal PTY lifecycle.

34.3 End-to-end tests

Use packaged or near-packaged builds to test:

First-run scan.

Workspace creation.

Add non-Git folder and initialise.

Create project.

Clone repository.

Switch workspace while processes run.

Open terminal and run Node.

Commit and push to a disposable test repository.

PR/check workflow in a controlled GitHub test organisation.

Installer cancellation and resume.

Application update flow.

34.4 Security tests

XSS cannot reach Node or IPC beyond allow-list.

IPC from untrusted frame is denied.

Branch/repository names containing shell metacharacters remain data.

Malicious package scripts do not run before trust.

Secret values are absent from logs, database, renderer events, and crash payloads.

Remote content cannot navigate privileged windows.

Symlink/path traversal cannot escape authorised operation roots.

34.5 Performance tests

Fixtures should include:

100,000-commit history.

Thousands of branches/tags.

Large working tree.

Very large diff.

High terminal output rate.

Multiple simultaneous repositories and PTYS.

34.6 Platform matrix

At minimum before stable release:

macOS Apple Silicon, current supported version.

macOS Intel if promised.

Windows 11 x64.

Oldest supported Windows build.

Fresh machine/user profile.

Existing developer machine with prior Git, Node, gh, and shell customisation.

35 Quality gates and definition of done

A feature is not done until:

Product behaviour matches the requirement ID.

Loading, empty, success, failure, cancellation, partial, and offline states are implemented where relevant.

IPC input and external output are runtime-validated.

Security review is complete for any new command, credential, web content, installer, or permission.

Unit and integration tests pass.

macOS and Windows behaviour is verified where the feature applies.

Accessibility is reviewed.

Logs and receipts are redacted.

Real data is used; mock data is limited to tests/storybook/demo mode.

Documentation and ADRs are updated.

Performance does not block the main process or renderer.

A recovery path exists for partial mutations.

36 Implementation programme

The team should build vertical slices that prove the real system, not a broad mock interface.

36.1 Phase 0: Technical risk spikes

36.1.1 P0-A: Electron/native baseline

Prove:

Electron packaging on macOS and Windows.

node-pty operation.

SQLite native module operation.

utility-process crash recovery.

signing pipeline placeholders.

Exit criteria: packaged test app runs a PTY and reads/writes the database on both platforms.

36.1.2 P0-B: Multi-account GitHub isolation

Prove two GitHub accounts across two workspace config roots on macOS and Windows. Test:

gh auth status.

repository listing.

Git credential helper.

clone/fetch/push.

token storage behaviour.

Exit criteria: ADR-002 selects the safe production strategy, including fallback token injection if needed.

36.1.3 P0-C: Environment runner

Prove deterministic environment merging, executable resolution, redaction, cancellation, and receipts.

Exit criteria: the same typed operation runs with two different workspace Git identities without modifying the normal global Git configuration.

36.1.4 P0-D: Git graph

Prove parser, pagination, stable lane algorithm, and virtualised rendering against a large generated repository.

36.2 Phase 1: Secure application foundation

Deliver:

Electron shell.

Strict preload/IPC.

Worker supervision.

SQLite and migrations.

Operation/receipt framework.

Workspace CRUD.

Project registration.

Security controls and logging.

Exit criteria: user can create and switch workspaces and add folders; no arbitrary command path exists from renderer.

36.3 Phase 2: Computer bootstrap

Deliver:

Tool manifest.

macOS scan and install plan.

Windows scan and install plan.

Git, gh, Volta, Node setup.

Verification and receipts.

resumable onboarding.

Exit criteria: clean supported machines can be prepared and interrupted setup can resume safely.

36.4 Phase 3: Project and terminal foundation

Deliver:

Real PTY terminal.

Project detection and trust.

Create/add/clone flows.

Runtime and package-manager detection.

Development process management.

Exit criteria: user can create or add a project, trust it, install dependencies, and run it from a workspace-bound terminal.

36.5 Phase 4: Complete local Git

Deliver:

Status, staging, commit.

Branches, refs, tags.

Fetch/pull/push.

Conflict states.

Full history graph.

Commit inspector.

Activity events.

Exit criteria: daily Git work can be completed without leaving Space, while every action remains a real Git operation.

36.6 Phase 5: GitHub workflow

Deliver:

Workspace GitHub auth.

Publish repository.

Clone/list repositories.

Issues.

PRs and checks.

Actions.

Releases.

Exit criteria: user can take a local project through publish, PR, checks, merge, and release in a disposable test organisation.

36.7 Phase 6: Intent and agent layer

Deliver:

Diff intent grouping.

Commit/PR/release writing assistance.

Typed agent actions.

Permission system.

File modifications and test execution.

Exit criteria: agent changes are traceable to files and operations, secrets are excluded, and high-impact actions require confirmation.

36.8 Phase 7: Automation and hardening

Deliver:

Local triggers and actions.

Cross-platform edge-case hardening.

Performance work.

accessibility completion.

updater and support bundles.

beta telemetry policy.

37 Epics and acceptance criteria

37.1 Epic E1: Create workspace

Acceptance:

User can create a workspace using only a name.

Internal UUID is stable across rename.

Workspace folders are created safely.

Switching changes projects, terminals, environment, GitHub, and activity context.

Running processes retain original context.

37.2 Epic E2: Bootstrap computer

Acceptance:

Scan is non-mutating.

Existing healthy tools are preserved.

Missing essentials are shown before installation.

Individual steps can fail and retry.

Verification occurs in a fresh constructed environment.

Receipts record every change.

Space never runs permanently elevated.

37.3 Epic E3: Add non-Git project

Acceptance:

Nested-repository risk is detected.

.gitignore is proposed based on actual files.

likely secrets are blocked from first commit.

Git initialises with selected default branch.

First real commit is created with workspace identity.

Publish-to-GitHub offer appears.

37.4 Epic E4: Publish to GitHub

Acceptance:

Uses active workspace account.

Owner/name/visibility are explicit.

Existing target collision is handled.

Same local repository receives remote and upstream.

Partial create/push state is recoverable.

Result is verified and receipted.

37.5 Epic E5: Git history

Acceptance:

Real commits and parent SHAs drive the graph.

Working-tree node and refs are accurate.
Merge curves remain stable during pagination.
View uses full workspace.
100,000-commit fixture remains navigable.
selection inspector shows exact technical detail.

37.6 Epic E6: Workspace account isolation

Acceptance:

Two workspaces can use different Git identities simultaneously.
Two terminals can remain open with distinct contexts.
GitHub accounts pass the P0 multi-account test.
Normal global user configurations are not rewritten during switching.
Mismatch warning appears before wrong-account remote actions.

37.7 Epic E7: Integrated terminal

Acceptance:

Real interactive shell works on macOS and Windows.
Workspace environment is visible to commands.
Authentication flows work.
High-volume output does not freeze renderer.
terminal remains bound to original workspace after switching.

37.8 Epic E8: Security baseline

Acceptance:

Renderer has no Node access.
CSP and navigation restrictions are tested.
No generic command IPC exists.
untrusted project scripts do not run automatically.
secret-redaction tests pass.
signing and packaged security checks pass.

38 UI state checklist

Every relevant screen must define these states before implementation approval:

Loading: Show what is being loaded and allow cancellation when possible.

Empty: Explain why it is empty and provide one clear next action.

Success: Show verified outcome, not only command completion.

Failure: Explain failed step, changed state, and recovery.

Partial: Preserve completed results and offer targeted continuation.

Offline: Preserve local features and label stale remote data.

Permission needed: Explain exact permission and reason.

Authentication needed: Launch service-native flow in workspace context.

Unsupported: Name the unsupported OS/tool/version and remediation.

Cancelled: Distinguish user cancellation from technical failure.

39 Prohibited implementation shortcuts

The team MUST NOT:

Build the Git graph from hard-coded lane fixtures in production.

Parse terminal-coloured human output when a stable machine-readable form exists.

Add runAnyCommand, generic shell IPC, or direct Node access to the renderer.

Store tokens in SQLite or plain JSON.

Rewrite global config files on every workspace switch.

Run dependency installs before trust.

Interpret installer exit code zero as complete verification.

hide partial side effects after remote failures.

silently force push, hard reset, delete branches, publish releases, or deploy production.

mix projects from different workspaces in one action without explicit ownership resolution.

install every optional tool during onboarding.

treat workspace isolation as a malicious-code sandbox.

depend on a backend for core local functionality.

ship unsigned public builds.

40 Architecture Decision Records required

40.1 ADR-001: Electron build and package toolchain

Decide exact Electron major, Forge/Vite configuration, native-module rebuild strategy, and workspace package manager.

40.2 ADR-002: GitHub multi-account credential strategy

Based on P0-B, decide GH_CONFIG_DIR, OS credential-store mapping, GH_TOKEN injection fallback, and Git credential helper behaviour.

40.3 ADR-003: SQLite driver and worker boundary

Decide driver, migration tool, backup mechanism, and native-module packaging.

40.4 ADR-004: Installer orchestration

Decide package-manager commands, official fallback installers, elevation strategy, download verification, and restart behaviour.

40.5 ADR-005: Workspace environment inheritance

Define exact allow-list/deny-list, PATH construction, proxy variables, locale, certificate variables, and child-process environment policy.

40.6 ADR-006: Project trust

Define trusted-path rules, downloaded-file indicators, script permissions, secret access, and re-prompt conditions.

40.7 ADR-007: Git history indexing and cache

Decide memory/disk cache, invalidation, pagination, and large-repository strategy.

40.8 ADR-008: Model provider and privacy

Decide local/remote model support, prompt content policy, redaction, retention, and opt-in.

41 Open product questions requiring explicit decisions

These questions must be resolved before the relevant phase, but they do not block foundation work:

Final product name and application identifiers.

Exact minimum supported macOS and Windows versions.

Whether macOS Intel is supported at stable launch.

Whether Windows ARM64 is included at launch.

Default editor recommendations.

Whether the app can install WSL or only detect/recommend it.

Whether workspace-level browser profiles are included in V1 or later.

Whether project creation uses built-in curated templates, an agent, or both.

Telemetry default and privacy policy.

Cloud-provider adapter order after GitHub.

Whether activity can be synchronised across computers.

Pricing and account requirements, if any; core local architecture must not require login unless product policy explicitly changes.

42 Technical references

The team should use current official documentation and pin exact versions in ADRs and build files. The following are the primary references for this specification:

42.1 Electron

Process model: <https://www.electronjs.org/docs/latest/tutorial/process-model>

Security: <https://www.electronjs.org/docs/latest/tutorial/security>

Context isolation: <https://www.electronjs.org/docs/latest/tutorial/context-isolation>

Utility processes: <https://www.electronjs.org/docs/latest/api/utility-process>

Safe storage: <https://www.electronjs.org/docs/latest/api/safe-storage>

Fuses: <https://www.electronjs.org/docs/latest/tutorial/fuses>

Updates: <https://www.electronjs.org/docs/latest/tutorial/updates>

42.2 Electron Forge

Documentation: <https://www.electronforge.io/>

macOS signing: <https://www.electronforge.io/guides/code-signing/code-signing-macos>

Windows signing: <https://www.electronforge.io/guides/code-signing/code-signing-windows>

42.3 Git

Git status porcelain formats: <https://git-scm.com/docs/git-status>

Git configuration and environment: <https://git-scm.com/docs/git-config>

Git log topology and parents: <https://git-scm.com/docs/git-log>

Ref enumeration: <https://git-scm.com/docs/git-for-each-ref>

42.4 GitHub CLI

Environment variables and GH_CONFIG_DIR: https://cli.github.com/manual/gh_help_environment

Authentication: https://cli.github.com/manual/gh_auth_login

Git credential setup: https://cli.github.com/manual/gh_auth_setup-git

Repository creation: https://cli.github.com/manual/gh_repo_create

PR checks: https://cli.github.com/manual/gh_pr_checks

Workflow dispatch: https://cli.github.com/manual/gh_workflow_run

Run watching: https://cli.github.com/manual/gh_run_watch

Releases: https://cli.github.com/manual/gh_release_create

42.5 Tool setup

Homebrew: <https://brew.sh/>

Homebrew Bundle: <https://docs.brew.sh/Brew-Bundle-and-Brewfile>

WinGet: <https://learn.microsoft.com/windows/package-manager/winget/>

Volta: <https://docs.volta.sh/guide/>

node-pty: <https://github.com/microsoft/node-pty>

xterm.js: <https://xtermjs.org/>

43 Final implementation directive

The first deliverable must not be a polished mock dashboard. It must be a secure vertical slice that proves the core promise:

Create two named workspaces.

Give each a distinct Git identity without rewriting the normal global Git config.

Open a real workspace-bound terminal in each.

Add a real project.

Initialise real Git when missing.

Create a real commit.

Render real history and branches from Git data.

Authenticate and publish through the workspace GitHub context.

Switch workspaces while both terminals remain correctly isolated.

Produce receipts for every mutating action.

Once that slice is reliable on macOS and Windows, the team can expand the interface, service adapters, GitHub automation, and agent capabilities without rebuilding the foundation.